



Databases and Big Data Project



Spotifv

Group Academic Project

Table of Contents

01. Objective



02. Dataset Info



03. ER Diagram



04. SQL



05. Python



06. Queries



07. Results



08. Conclusion



09. Thanks!



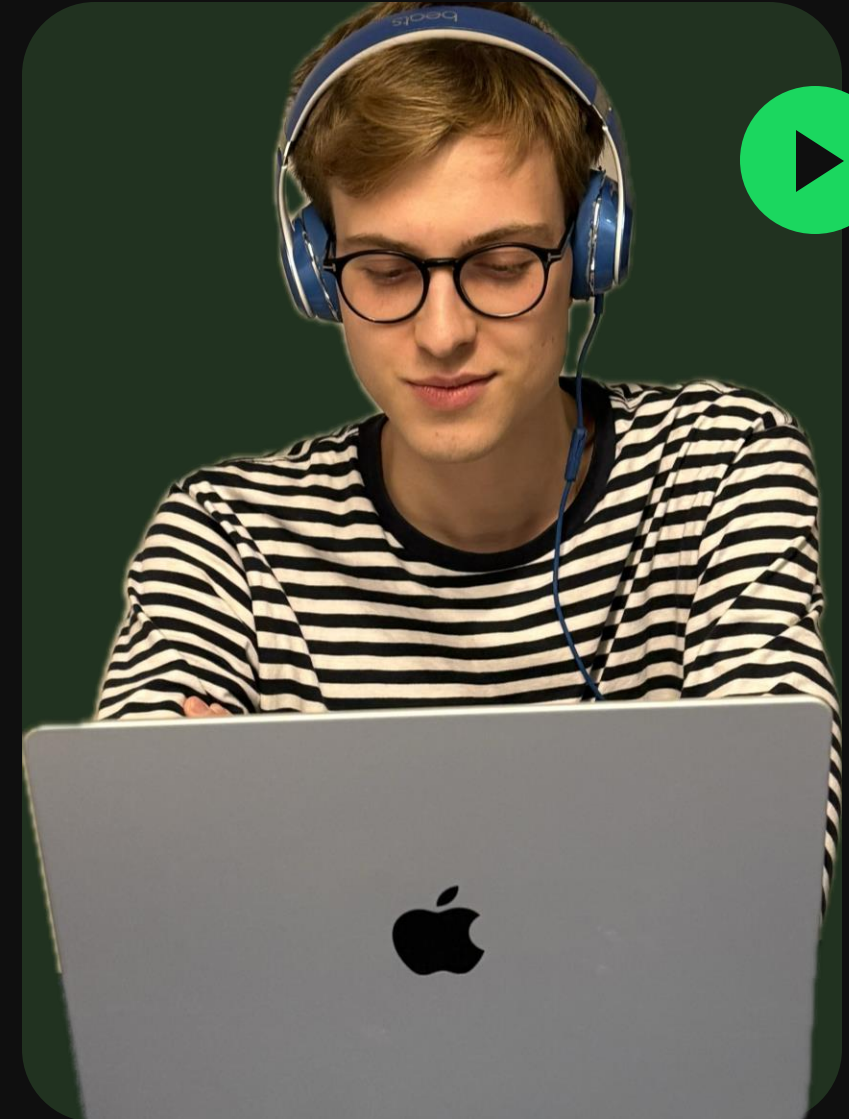


Introduction to the Dataset

FEATURE	DESCRIPTION
Total tracks	953
Attributes	25 columns
Platforms covered	4
Time range	2010 - 2024
Key metrics	11

Key Features:

- **Track Details:** Includes song titles, artists, release dates, and chart placements.
- **Streaming Metrics:** Aggregated data on Spotify, Apple, and Deezer playlists and charts.
- **Musical Attributes:** Key musical components such as BPM, danceability, valence, energy, and acousticness.





ER DIAGRAM

Visual representation of the entities, relationships, and attributes in a database system

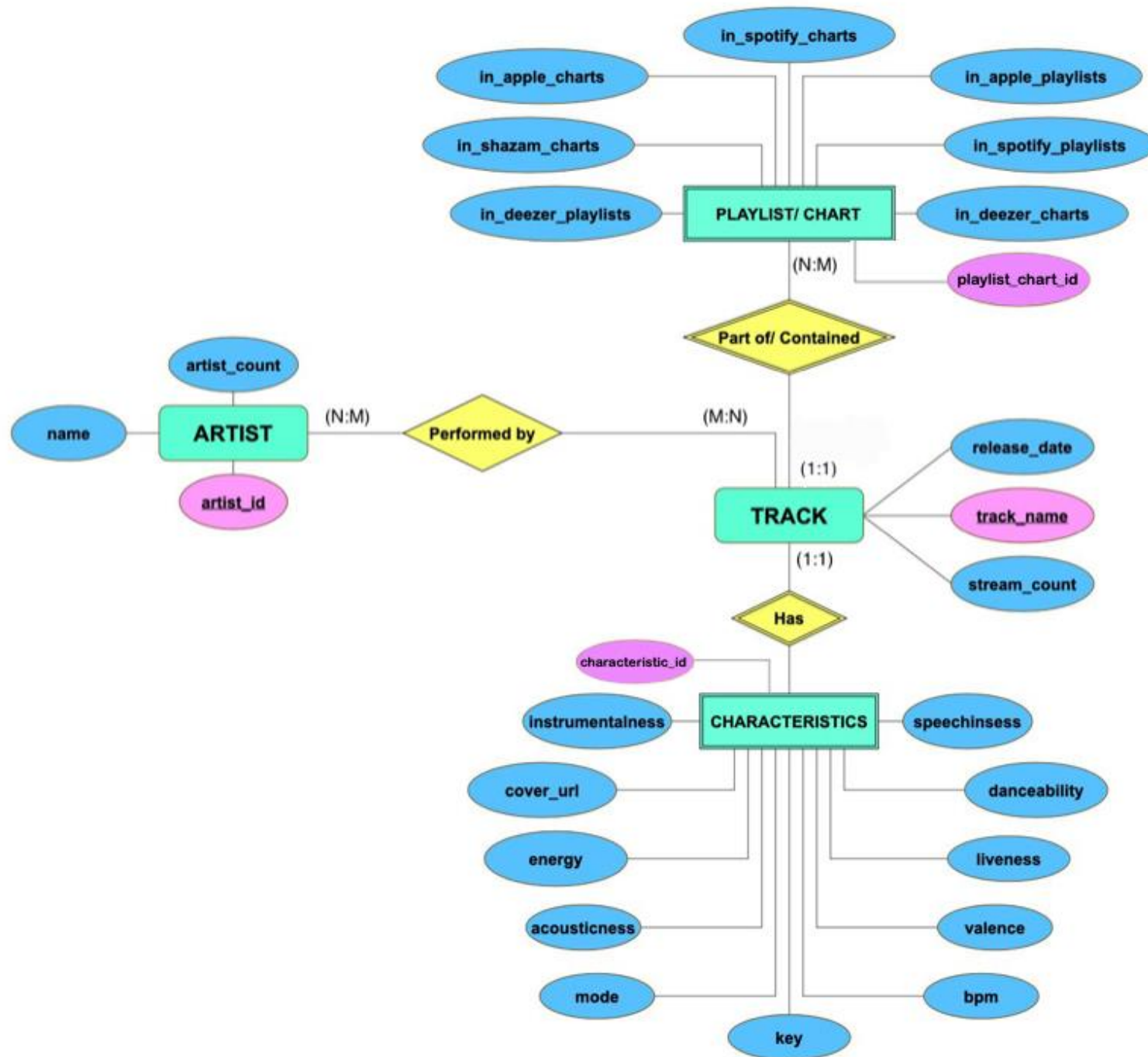


It is used in database design to **map the structure** of data and its relationships.



Click [here](#) to visualize the ER DIAGRAM







SQL



```
CREATE TABLE if not exists Track (  
  track_id VARCHAR(255) PRIMARY KEY,  
  track_name VARCHAR(255) NOT NULL,  
  release_date DATE NOT NULL,  
  stream_count BIGINT,  
  artist_id VARCHAR(255) NOT NULL,  
  FOREIGN KEY (artist_id) REFERENCES Artist(artist_id)  
);
```

- The Playlist_Chart table keeps track of how songs perform on different platforms.
- Both tables are connected through the track_id column, ensuring every chart record corresponds to a valid song in the Track table.

- The Track table stores information about songs.
- It connects to an Artist table to ensure every song is linked to a valid artist.

```
CREATE TABLE if not exists Playlist_Chart (  
  playlist_chart_id VARCHAR(255) PRIMARY KEY,  
  in_spotify_playlists INT,  
  in_spotify_charts INT,  
  in_apple_playlists INT,  
  in_apple_charts INT,  
  in_deezer_playlists INT,  
  in_deezer_charts INT,  
  in_shazam_charts INT,  
  track_id VARCHAR(255) NOT NULL,  
  FOREIGN KEY (track_id) REFERENCES Track(track_id)  
);
```



SQL



```
CREATE TABLE if not exists Artist (  
  artist_id VARCHAR(255) PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  artist_count INT  
);
```

- The Characteristics table captures detailed musical attributes for each track and it also includes a link to the track's cover image.
- This table links to the Track table through the track_id, ensuring each set of attributes corresponds to a valid track.

- The Artist table stores information about artists.
- It connects to the Track table to associate each song with its respective artist.

```
CREATE TABLE if not exists Characteristics (  
  characteristic_id VARCHAR(255) PRIMARY KEY,  
  track_id VARCHAR(255) NOT NULL,  
  bpm INT,  
  `key` VARCHAR(5),  
  mode VARCHAR(10),  
  danceability DECIMAL(5, 2),  
  valence DECIMAL(5, 2),  
  energy DECIMAL(5, 2),  
  acousticness DECIMAL(5, 2),  
  instrumentalness DECIMAL(5, 2),  
  liveness DECIMAL(5, 2),  
  speechiness DECIMAL(5, 2),  
  cover_url TEXT,  
  FOREIGN KEY (track_id) REFERENCES Track(track_id)  
);
```



Python implementation

- This establishes a connection to MySQL database using the pymysql library.
- Database connection credential are specified to link to the correct database.



```
import pymysql
import pandas as pd
import numpy as np

# Database connection
connection = pymysql.connect(
    host="localhost",
    user="root",
    password="Giacomo00",
    database="spotify_most_streamed_songs"
)
```

```
try:
    # Read the dataset
    file_path = "Spotify Most Streamed Songs.csv"
    data = pd.read_csv(file_path)

    # Replace NaN with None for compatibility with MySQL
    data = data.replace({np.nan: None})

    # Load data into the 'Artist' table
    with connection.cursor() as cursor:
        for index, row in data.iterrows():
            # Insert into Artist table
            cursor.execute("""
                INSERT INTO Artist (artist_id, name, artist_count)
                VALUES (%s, %s, %s)
                ON DUPLICATE KEY UPDATE name=name;
            """, (index, row['artist(s)_name'], row['artist_count']))
```

- Reads the CSV file using pandas.
- Missing values (NaN) in the dataset are replaced with None for compatibility with MySQL.



Script inserts data into 4 primary tables:

ARTIST TABLE:

- Stores information about artist.
- Uses a loop to insert or update artist data for each row in the dataset.





Python implementation

- This python scripts cleans numeric columns by replacing NaN with 0 and removing commas, then converts them to integers.
- Replaces other NaN values with None for MySQL compatibility.



```
# Clean numeric columns with potential commas or NaN values
columns_to_clean = [
    'in_spotify_playlists', 'in_spotify_charts', 'in_apple_playlists',
    'in_apple_charts', 'in_deezer_playlists', 'in_deezer_charts', 'in_shazam_charts'
]

for col in columns_to_clean:
    data[col] = (
        data[col]
        .replace({np.nan: 0}) # Replace NaN with 0
        .astype(str) # Ensure the column is treated as strings
        .str.replace(',', '') # Remove commas
        .astype(int) # Convert to integers
    )

# Replace remaining NaN values in other columns with None
data = data.replace({np.nan: None}) # Replace remaining NaN with None for MySQL compatibility

# Debug output to verify dataset structure and cleaned data
print(data.head(300))
```



Python implementation

TRACK TABLE:

- Stores information about individual songs.
- The release_date is created by combining the year, month, and day columns from the dataset.

```
# Commit after inserting Track data
connection.commit()

# Load data into the 'Characteristics' table
with connection.cursor() as cursor:
    for index, row in data.iterrows():
        # Insert into Characteristics table
        cursor.execute("""
            INSERT INTO Characteristics (
                characteristic_id, track_id, bpm, `key`, mode, danceability, valence, energy,
                acousticness, instrumentalness, liveness, speechiness, cover_url
            ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
            ON DUPLICATE KEY UPDATE bpm=bpm;
        """, (
            index,
            index,
            row['bpm'],
            row['key'],
            row['mode'],
            row['danceability_%'],
            row['valence_%'],
            row['energy_%'],
            row['acousticness_%'],
            row['instrumentalness_%'],
            row['liveness_%'],
            row['speechiness_%'],
            row['cover_url']
        ))
```

```
# Commit after inserting Artist data
connection.commit()

# Load data into the 'Track' table
with connection.cursor() as cursor:
    for index, row in data.iterrows():
        # Insert into Track table
        cursor.execute("""
            INSERT INTO Track (track_id, track_name, release_date, stream_count, artist_id)
            VALUES (%s, %s, %s, %s, %s)
            ON DUPLICATE KEY UPDATE track_name=track_name;
        """, (
            index,
            row['track_name'],
            f"{row['released_year']}-{row['released_month']}-{row['released_day']}" if row['released_year'] and row['released_month'] and row['released_day'] else None,
            row['streams'],
            index # Assuming one artist per track
        ))
```

CHARACTERISTICS TABLE:

- Stores song features like bpm, energy, danceability, and valence.
- Includes additional metrics such as acousticness, instrumentalness, and liveness.



Python implementation

PLAYLIST TABLE:

- Tracks the presence of songs in various playlists and charts.
- Records the count of appearances in specific categories like playlists and charts for each platform.



```
# Commit after inserting Playlist_Chart data
connection.commit()

except Exception as e:
    print(f"An error occurred: {e}")
finally:
    connection.close()
```

```
# Load data into the 'Playlist_Chart' table
with connection.cursor() as cursor:
    for index, row in data.iterrows():
        # Insert into Playlist_Chart table
        cursor.execute("""
            INSERT INTO Playlist_Chart (
                playlist_chart_id, in_spotify_playlists, in_spotify_charts, in_apple_playlists,
                in_apple_charts, in_deezer_playlists, in_deezer_charts, in_shazam_charts, track_id
            ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
            ON DUPLICATE KEY UPDATE in_spotify_playlists=in_spotify_playlists;
        """, (
            index,
            row['in_spotify_playlists'],
            row['in_spotify_charts'],
            row['in_apple_playlists'],
            row['in_apple_charts'],
            row['in_deezer_playlists'],
            row['in_deezer_charts'],
            row['in_shazam_charts'],
            index
        ))
```

- Uses try-except blocks to catch and handle errors.
- Ensures the database connection is properly closed in a finally block





QUERIES



QUERY 1

Most Streamed Tracks

Retrieves the 10 tracks with the highest number of streams from the dataset.

```
queries = {  
  1: {  
    "description": "Most Streamed Tracks",  
    "query": ""  
      SELECT track_name, stream_count  
      FROM Track  
      ORDER BY stream_count DESC  
      LIMIT 10;  
    ""  
  }  
}
```



RESULTS



QUERY 2

Most Popular Artists

Identifies the top 5 artists with the highest total stream count across all their tracks.

```
2: {  
  "description": "Most Popular Artists",  
  "query": ""  
    SELECT Artist.name, SUM(Track.stream_count) AS total_streams  
    FROM Artist  
    JOIN Track ON Artist.artist_id = Track.artist_id  
    GROUP BY Artist.name  
    ORDER BY total_streams DESC  
    LIMIT 5;  
  ""  
}
```



RESULTS



QUERY 3

Average Song Characteristics

Calculates the average BPM, energy, and danceability across all tracks.

```
3: {  
  "description": "Average Song Characteristics",  
  "query": ""  
    SELECT  
      AVG(bpm) AS avg_bpm,  
      AVG(energy) AS avg_energy,  
      AVG(danceability) AS avg_danceability  
    FROM Characteristics;  
  ""  
}
```



RESULTS





QUERIES



QUERY 4

Tracks in Multiple Playlists/Charts

Find the tracks that appear in the most Spotify playlists or charts.

```
4: {
  "description": "Tracks in Multiple Playlists/Charts",
  "query": ""
  SELECT Track.track_name,
         Playlist_Chart.in_spotify_playlists + Playlist_Chart.in_spotify_charts AS total_appearances
  FROM Track
  JOIN Playlist_Chart ON Track.track_id = Playlist_Chart.track_id
  ORDER BY total_appearances DESC
  LIMIT 10;
  ""
}
```



RESULTS



QUERY 5

Tracks by Release Year

Count how many tracks were released each year.

```
5: {
  "description": "Tracks by Release Year",
  "query": ""
  SELECT YEAR(release_date) AS release_year, COUNT(*) AS track_count
  FROM Track
  GROUP BY release_year
  ORDER BY release_year DESC;
  ""
}
```



RESULTS



QUERY 6

Energy Distribution by Mode

Compare the average energy of tracks in Major vs. Minor mode.

```
6: {
  "description": "Energy Distribution by Mode",
  "query": ""
  SELECT mode, AVG(energy) AS avg_energy
  FROM Characteristics
  GROUP BY mode;
  ""
}
```



RESULTS





QUERIES



QUERY 7

Most Danceable Tracks

Identify the top 5 tracks with the highest danceability score.

```
7: {
  "description": "Most Danceable Tracks",
  "query": ""
  SELECT Track.track_name, Characteristics.danceability
  FROM Track
  JOIN Characteristics ON Track.track_id = Characteristics.track_id
  ORDER BY Characteristics.danceability DESC
  LIMIT 5;
  ""
}
```



RESULTS



QUERY 8

Tracks with the Highest Valence (Happiest Tracks)

Find the tracks with the highest valence score, indicating happiness or positivity.

```
8: {
  "description": "Tracks with the Highest Valence (Happiest Tracks)",
  "query": ""
  SELECT Track.track_name, Characteristics.valence
  FROM Track
  JOIN Characteristics ON Track.track_id = Characteristics.track_id
  ORDER BY Characteristics.valence DESC
  LIMIT 5;
  ""
}
```



RESULTS



QUERY 9

Tracks with Multiple Artists and Average Streams

Finds the tracks with the highest average stream count among those created by artists with more than one contributor.

```
9: {
  "description": "Tracks with Multiple Artists and Average Streams",
  "query": ""
  SELECT
    subquery.track_name AS "Song Title",
    subquery.artist_count AS "Number of Artists",
    AVG(Track.stream_count) AS "Average Streams"
  FROM (
    SELECT Track.track_name, Artist.artist_count, Track.stream_count
    FROM Track
    JOIN Artist ON Track.artist_id = Artist.artist_id
    WHERE artist_count > 1
  ) AS subquery
  JOIN Track ON subquery.track_name = Track.track_name
  GROUP BY subquery.track_name, subquery.artist_count
  ORDER BY "Average Streams" DESC
  LIMIT 5;
  ""
}
```



RESULTS

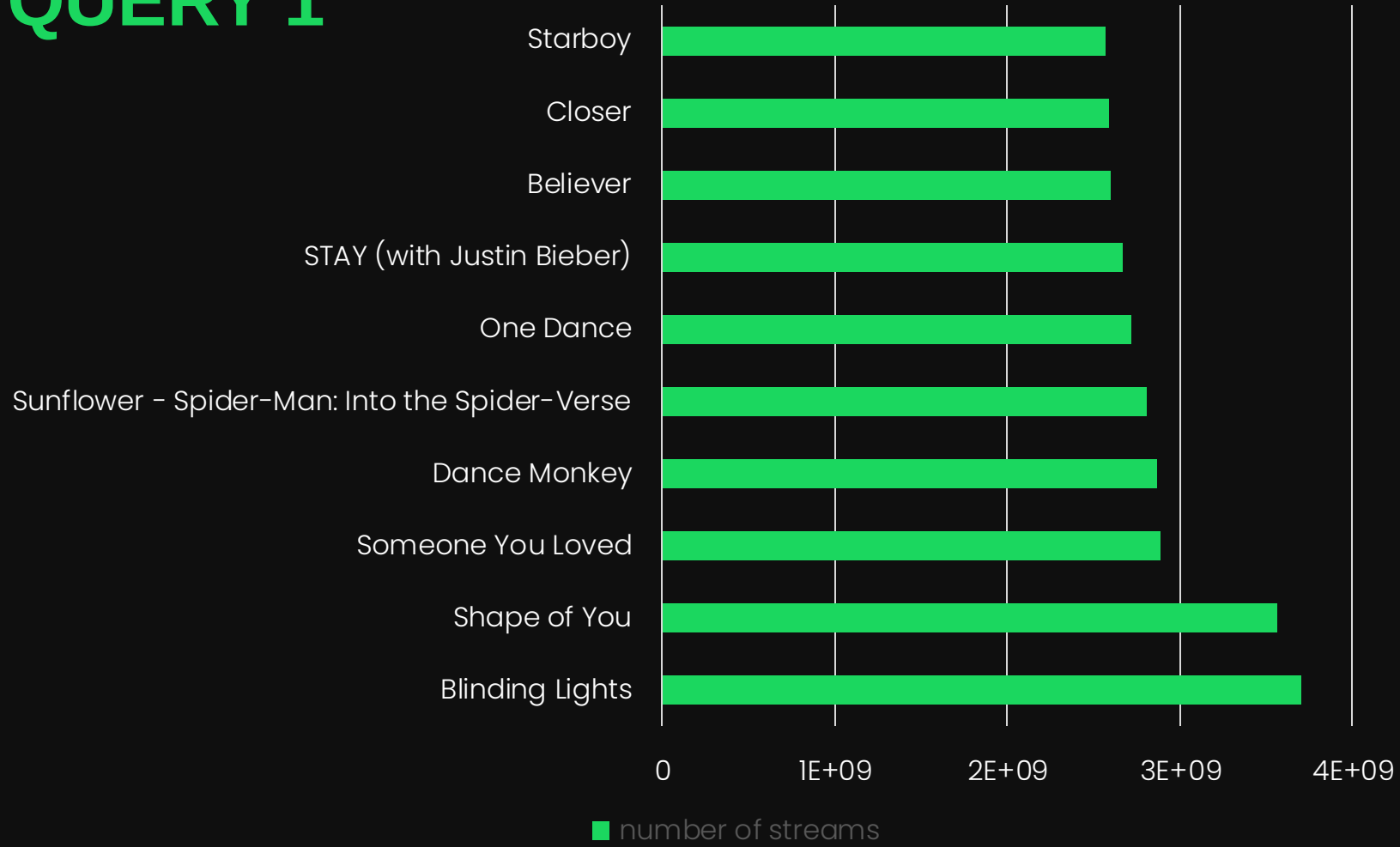




QUERY 1



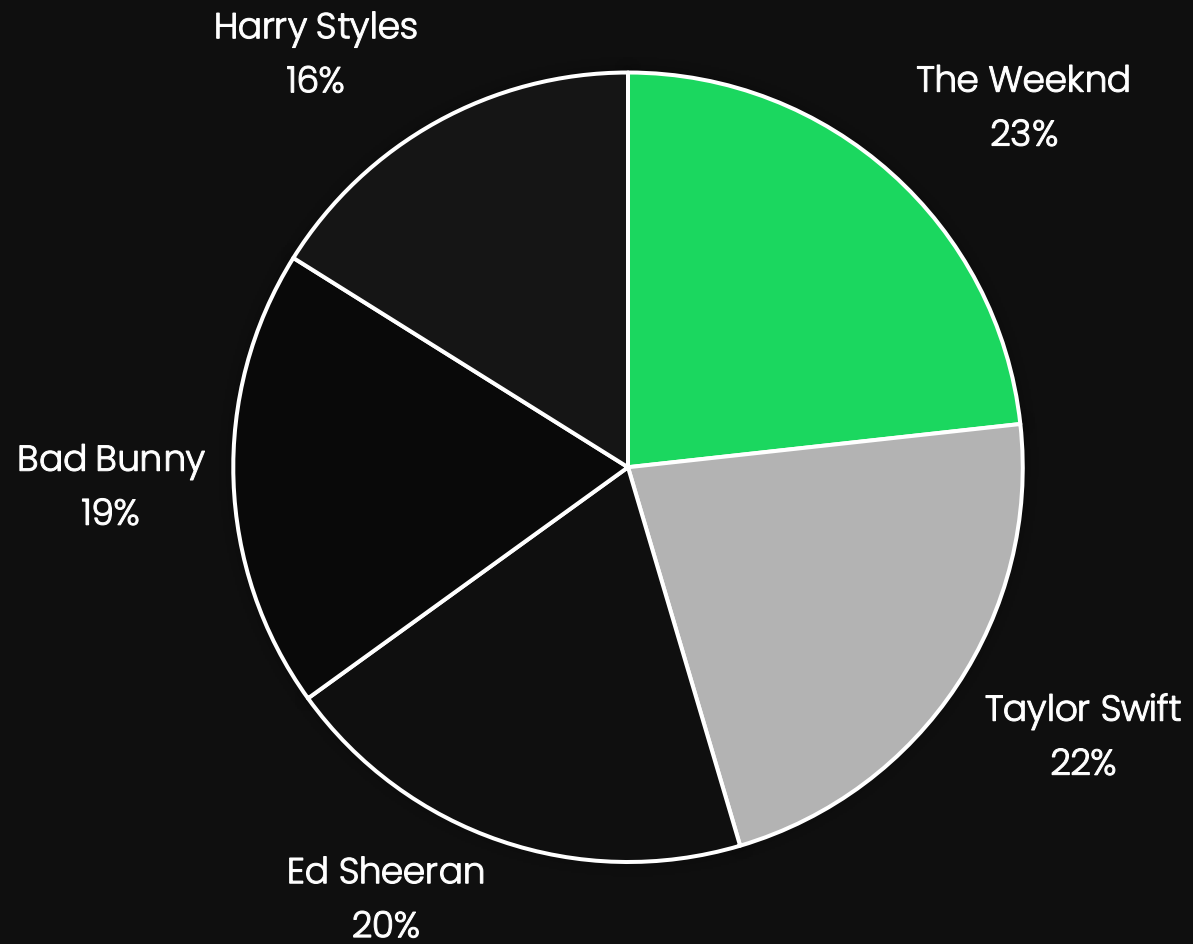
NUMBER OF STREAMS





QUERY 2

STREAM COUNT

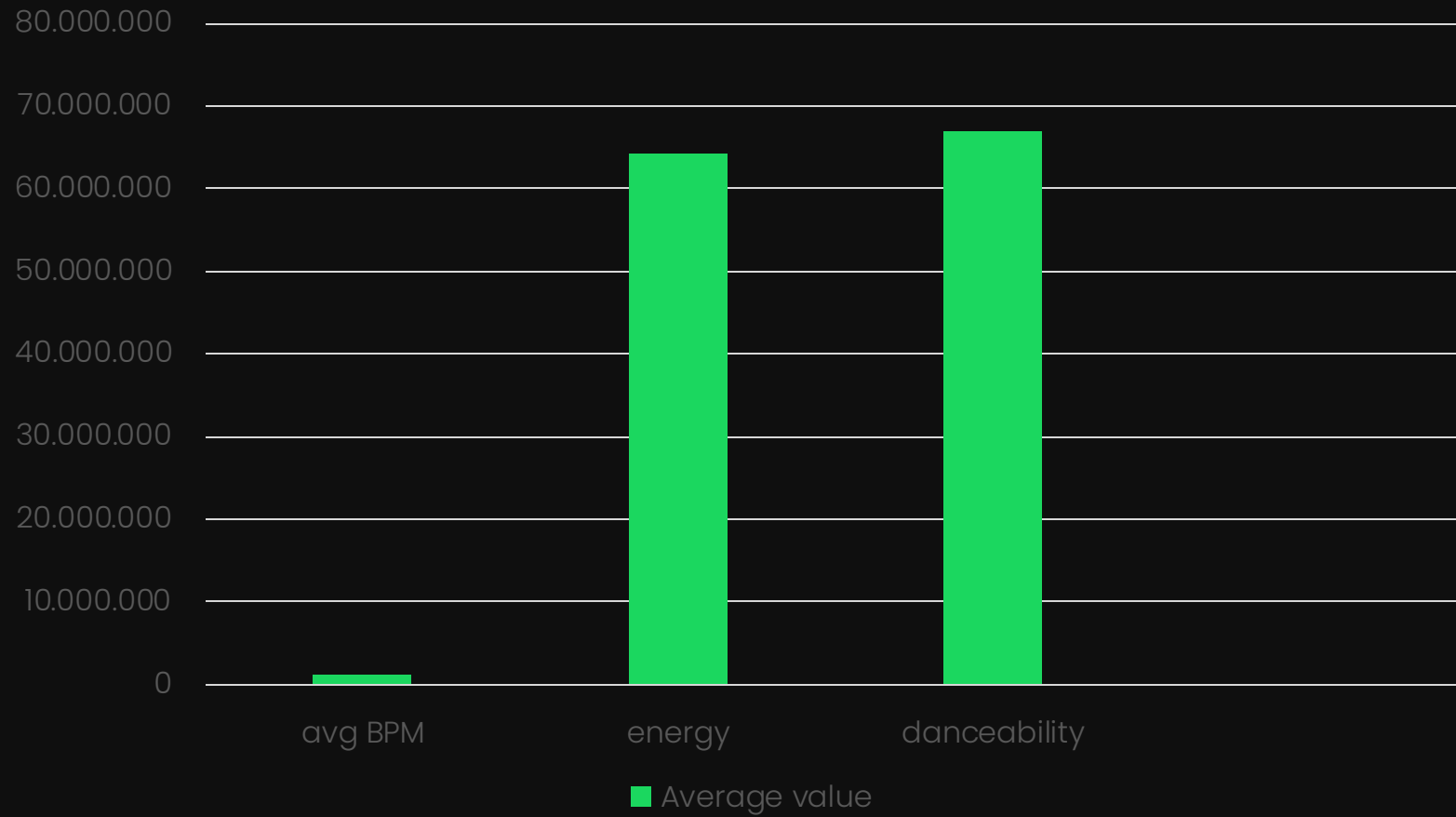




QUERY 3



CHARACTERISTICS

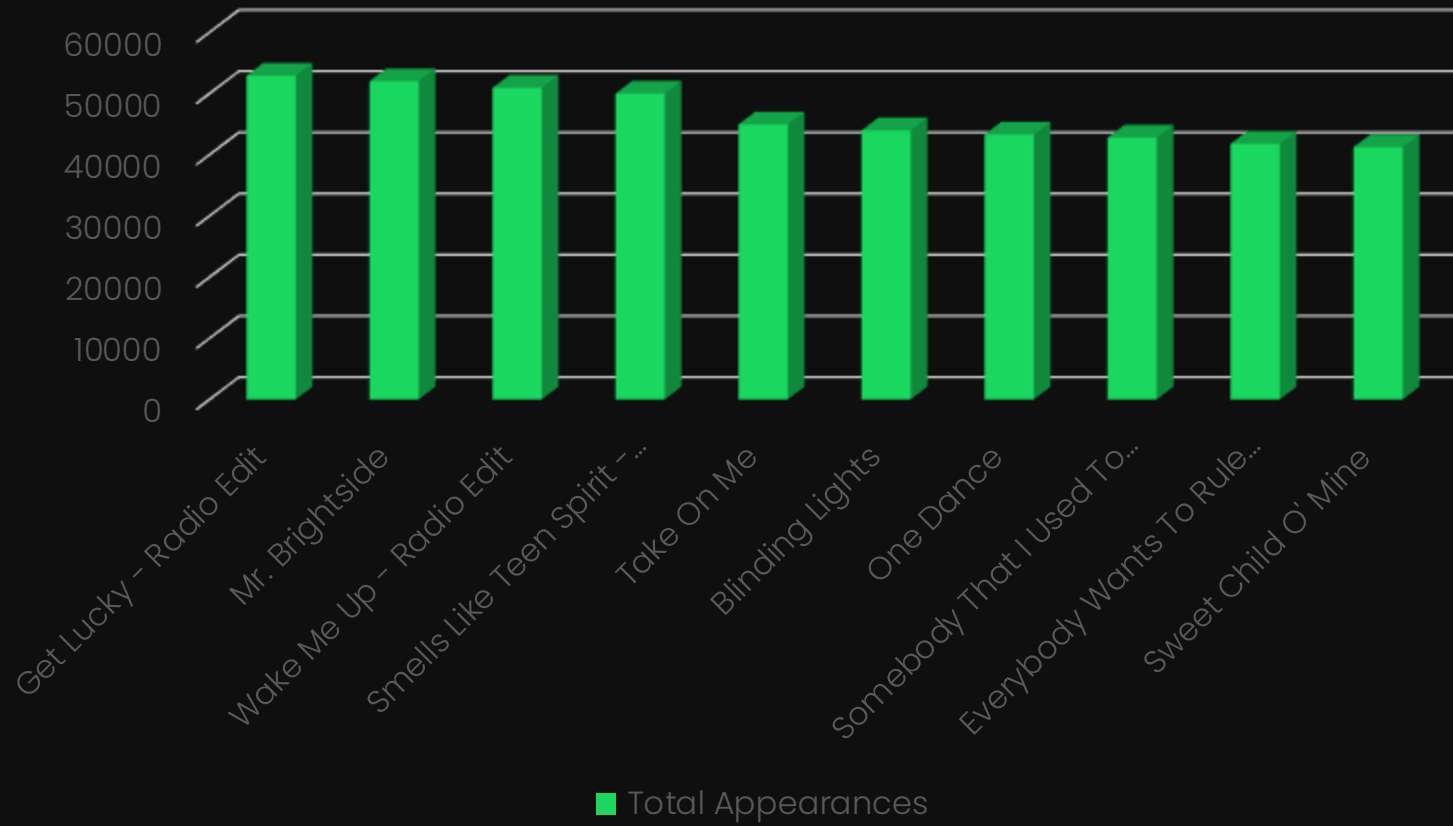




QUERY 4



TOTAL APPEARANCES

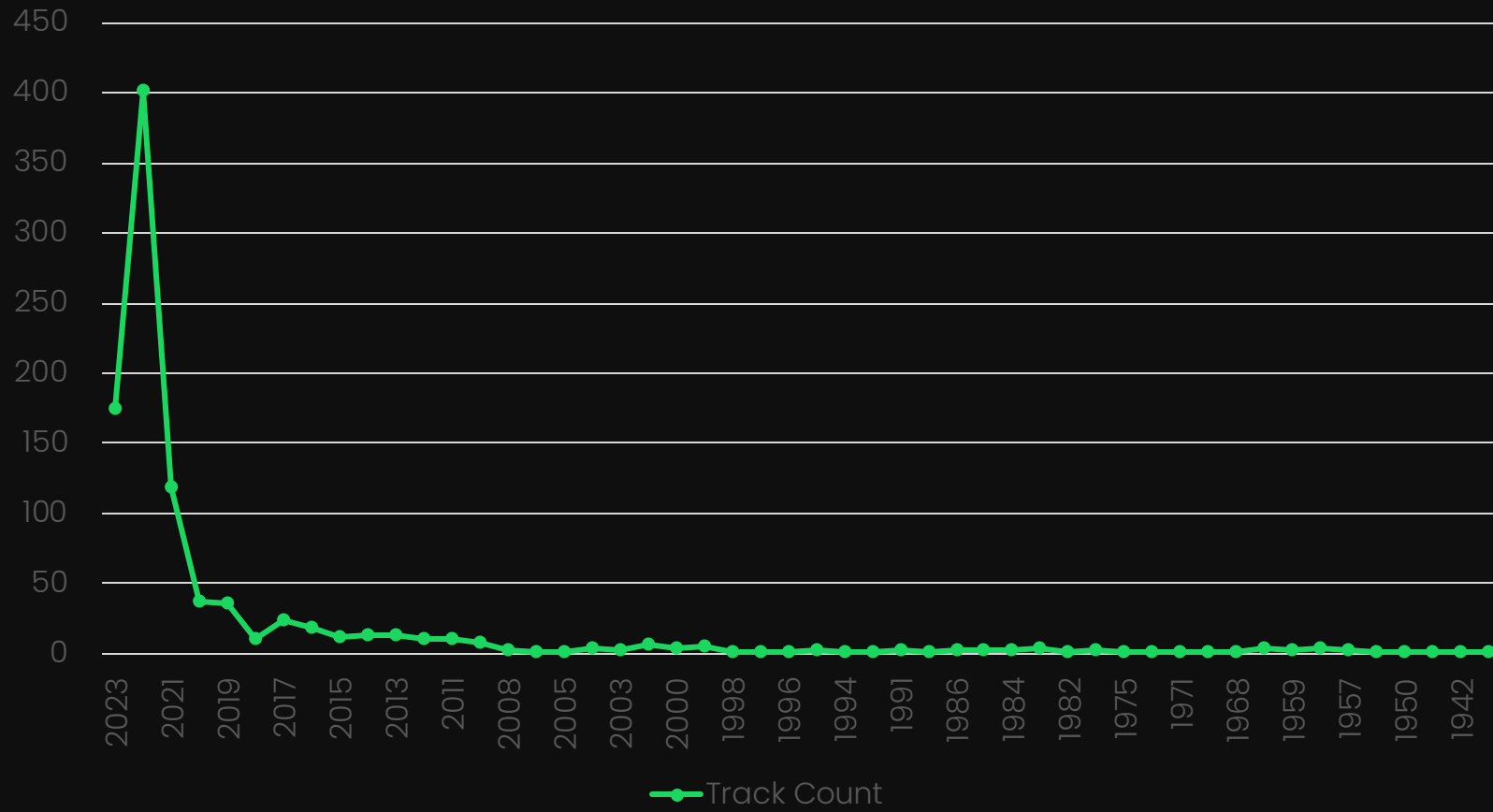




QUERY 5



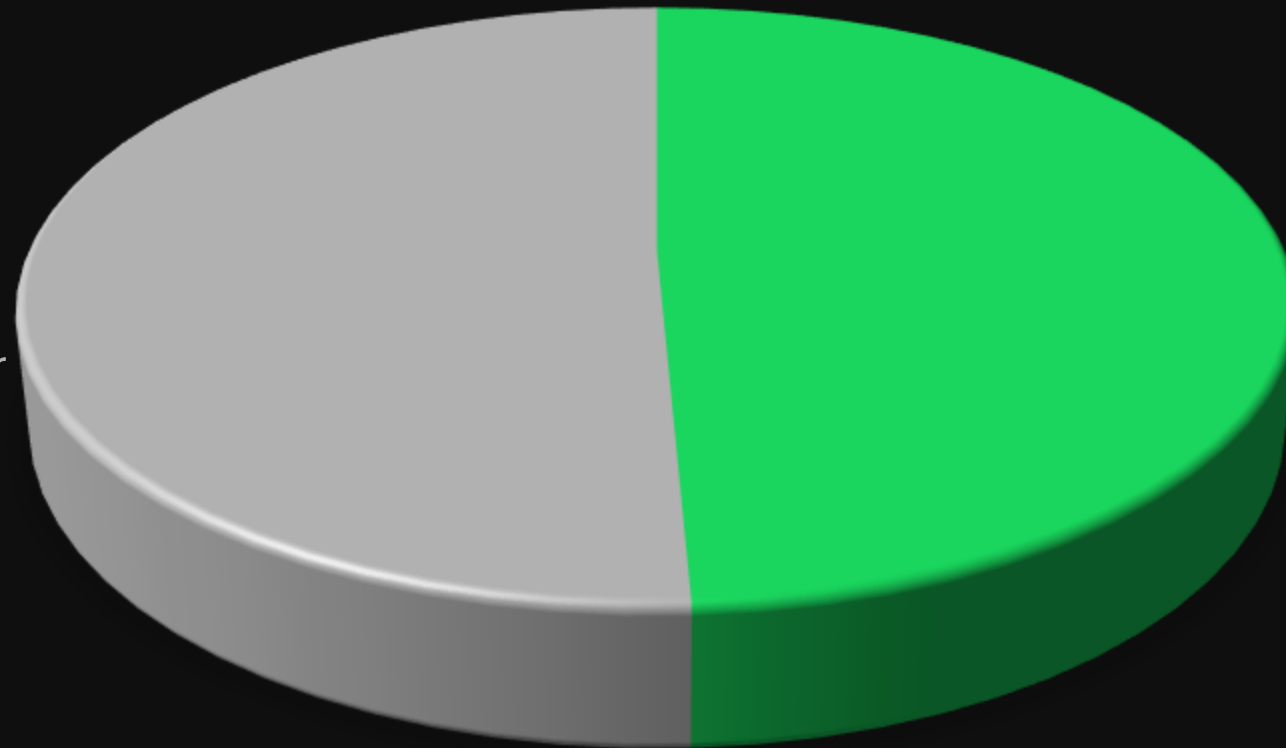
TRACK COUNT





QUERY 6

AVERAGE ENERGY



Minor
51%

Major
49%

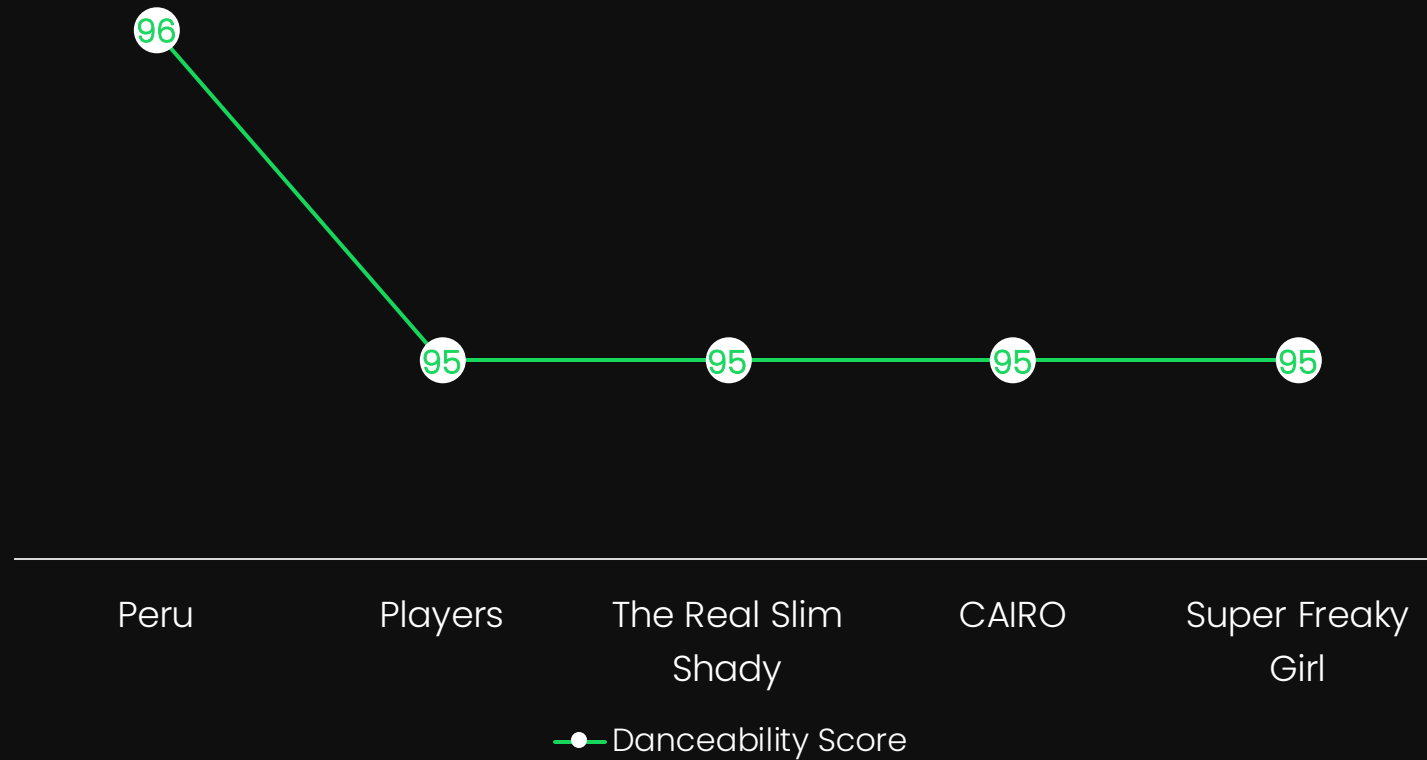




QUERY 7



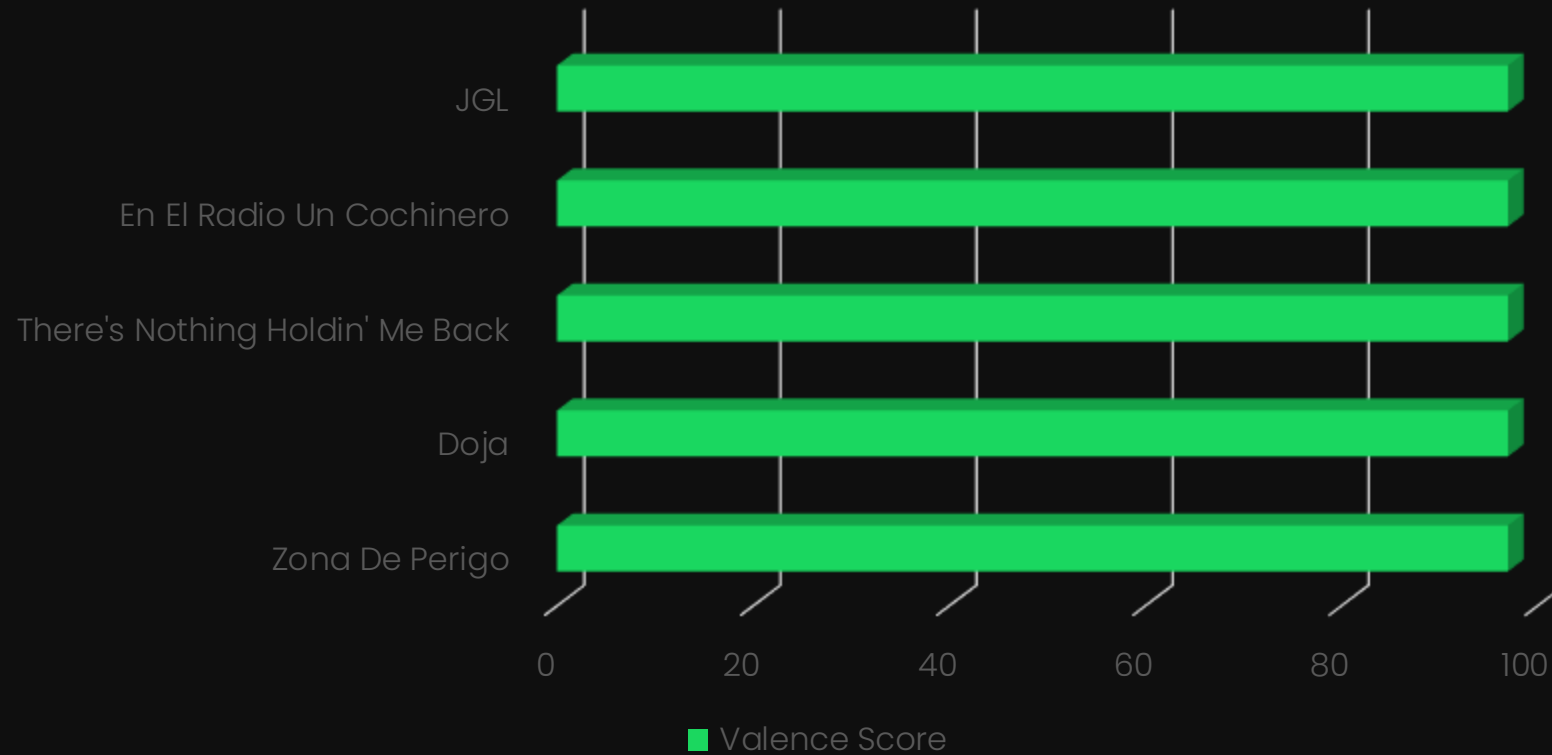
DANCEABILITY SCORE





QUERY 8

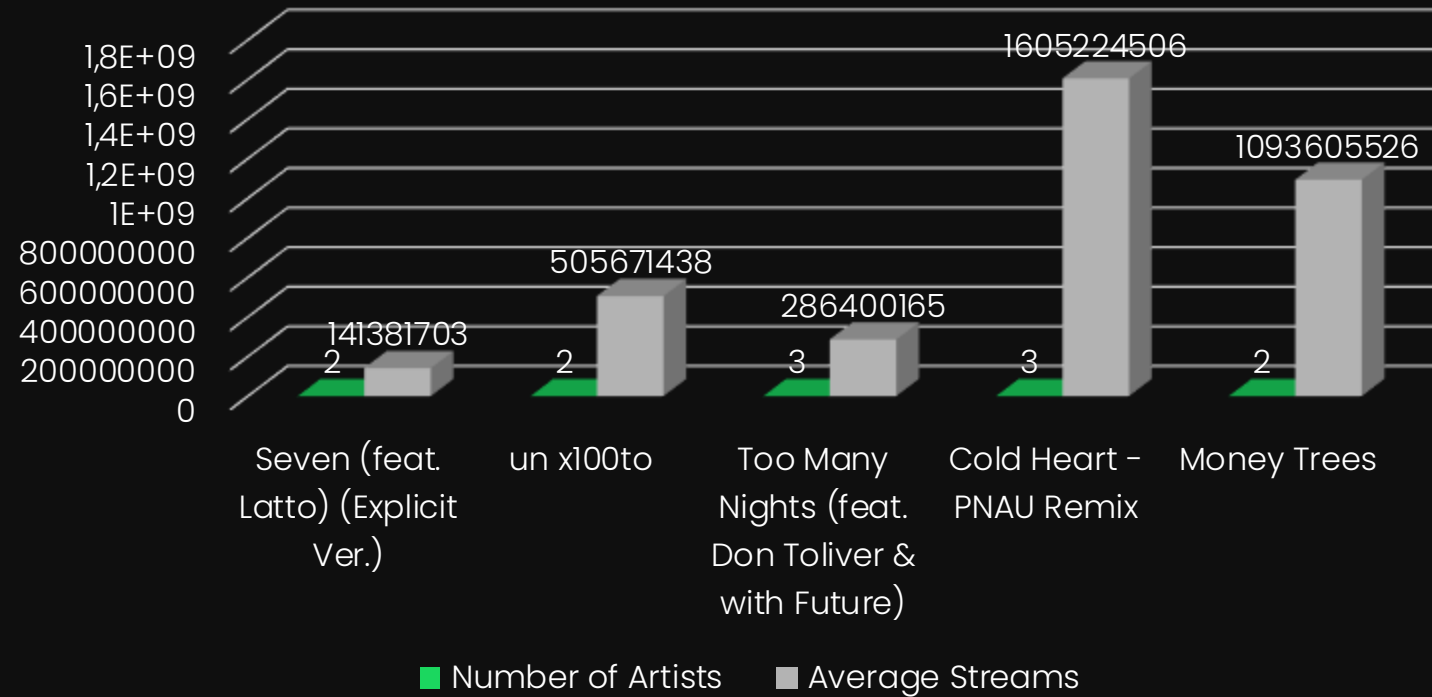
TRACKS WITH THE HIGHEST VALENCE
(Happiest tracks)





QUERY 9

TRACKS WITH MULTIPLE ARTISTS AND AVERAGE
STREAMS





**“Where words fail, music speaks”
— Hans Christian Andersen**

“ I think music in itself is healing. It's an explosive expression of humanity. It's something we are all touched by. No matter what culture we're from, everyone loves music.”

— Billy Joel



Thanks!